

Docker Fundamentals

An Introduction to Docker + Four Hands-On Labs on AWS EC2

From core concepts to building images, running multi-container apps, and persisting data — all inside an EC2 instance launched from the AWS Console.

Part 1 — Introduction to Docker

What Is Docker?

Docker is a platform for packaging an application together with everything it needs to run — code, runtime, system libraries, and configuration — into a single portable unit called a container. Because a container carries its own environment, it behaves identically on a laptop, a test server, or an AWS EC2 instance.

Docker solves the classic "it works on my machine" problem: instead of installing software directly on a machine and hoping the environment matches production, you build an image once and run it anywhere a Docker engine is installed.

Containers vs. Virtual Machines

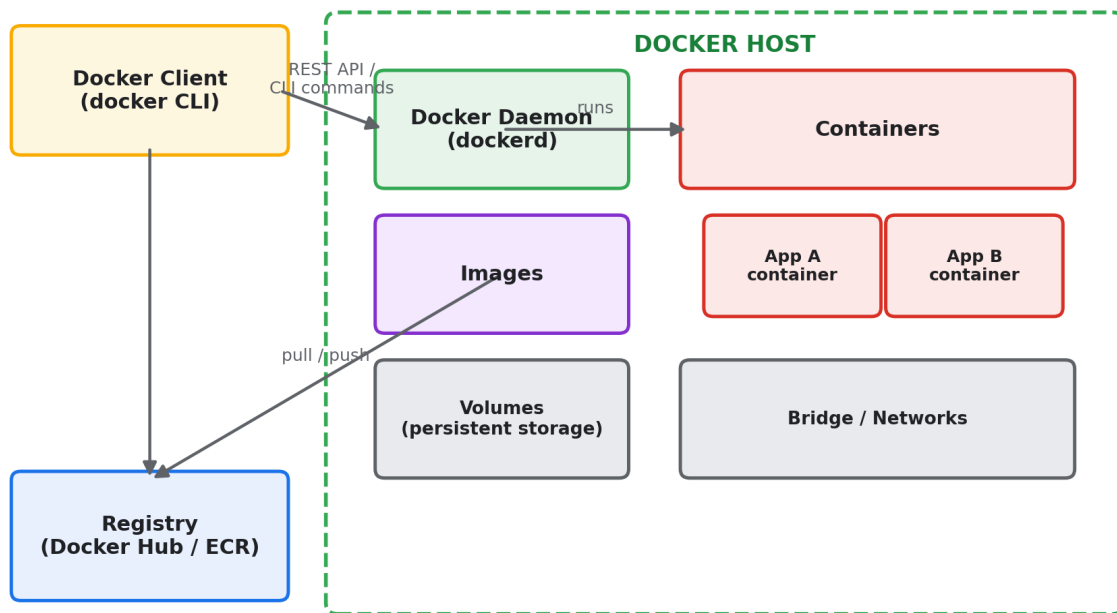
Virtual Machines	Docker Containers
Each VM includes a full guest operating system	Containers share the host machine's OS kernel
Heavier — gigabytes in size, slower to boot (minutes)	Lightweight — megabytes in size, starts in seconds
Managed by a hypervisor (e.g. VMware, KVM)	Managed by the Docker Engine (daemon)
Strong isolation, higher resource overhead	Process-level isolation, minimal overhead

Key Concepts

- Image — a read-only template with the app, its dependencies, and run instructions. Built from a Dockerfile.
- Container — a running instance of an image. You can start, stop, move, or delete containers.
- Dockerfile — a text file of build instructions (base image, copy files, install packages, set start command).
- Docker Engine (dockerd) — the background service on the host that builds images and runs containers.
- Registry — image storage/distribution, e.g. Docker Hub or Amazon ECR.
- Volume — a mechanism for persisting container data independent of the container's lifecycle.
- Docker Compose — a tool for defining and running multi-container applications from one YAML file.

How Docker Works — Architecture

Docker uses a client-server architecture. These pieces work together every time you run a docker command:



1. You type a command in the Docker Client (CLI), e.g. `docker run nginx`.
2. The client sends the request to the Docker Daemon (dockerd) over a REST API.
3. The daemon checks if the image exists locally; if not, it pulls it from a Registry.
4. The daemon creates a Container from the image — an isolated process using Linux namespaces and cgroups.
5. Containers can attach Volumes for storage and join Networks to communicate.

Because containers share the host kernel, many can run side-by-side on one EC2 instance far more efficiently than the same number of full VMs.

Part 2 — Environment Setup (Do Once)

All four labs below reuse the same EC2 instance. Set it up once here, then work through each lab in order.

Launch an EC2 Instance

6. Sign in to the AWS Console and open the EC2 service.
7. Click Launch instance and name it docker-lab-instance.
8. AMI: choose Amazon Linux 2023.
9. Instance type: t2.micro or t3.micro (free-tier eligible).
10. Key pair: create or select one (only needed for terminal SSH — EC2 Instance Connect works without it).
11. Network settings: allow inbound SSH (22) from your IP, and HTTP (80) and a custom TCP range 8080-8081 for the labs below.
12. Keep default storage (8 GiB) and click Launch instance.

Connect and Install Docker

Once the instance is Running, select it in the console, click Connect, choose the EC2 Instance Connect tab, and click Connect to open a browser-based terminal.

```
sudo yum update -y
sudo yum install -y docker
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
```

Note: log out and reconnect (or run `newgrp docker`) after `usermod` so your session picks up `docker-group` permissions without `sudo`.

```
docker --version
docker info
```

Confirms the Docker Engine installed and is running correctly.

Lab 1 — Run Your First Containers

Goal: confirm Docker works end-to-end and run a real web server.

```
docker run hello-world
```

Docker pulls the image (if not local), creates a container, runs it, prints a confirmation, then exits.

```
docker run -d --name my-nginx -p 80:80 nginx
```

- `-d` — run in the background (detached)
- `--name my-nginx` — friendly container name
- `-p 80:80` — map host port 80 to container port 80

13. Confirm the security group allows inbound HTTP (80).

14. Open `http://<ec2-public-ip>` in your browser — you should see the Nginx welcome page.

```
docker ps          # running containers
docker logs my-nginx # container logs
docker stop my-nginx # stop it
docker rm my-nginx  # remove it
```

Lab 2 — Build a Custom Image with a Dockerfile

Goal: package your own simple web page into an image instead of using the stock Nginx page.

15. Create a project folder and page:

```
mkdir ~/my-webapp && cd ~/my-webapp
```

```
cat > index.html << 'EOF'
<html>
<head><title>My Docker App</title></head>
<body><h1>Hello from my custom Docker image!</h1></body>
</html>
EOF
```

16. Create the Dockerfile:

```
cat > Dockerfile << 'EOF'
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
EOF
```

17. Build the image:

```
docker build -t my-webapp:v1 .
```

18. Run a container from your new image on a different port:

```
docker run -d --name webapp-container -p 8080:80 my-webapp:v1
```

19. Open the security group's port range and browse to:

```
http://<ec2-public-ip>:8080
```

You should see your custom page, proving the image you built — not the base Nginx page — is being served.

```
docker images          # see my-webapp:v1 listed
docker history my-webapp:v1 # see the layers you built
```

Lab 3 — Multi-Container App with Docker Compose

Goal: run two connected containers (a web app and a Redis cache) defined declaratively in one file.

20. Install Docker Compose plugin (Amazon Linux 2023 usually needs this separately):

```
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL
https://github.com/docker/compose/releases/latest/download/docker-compose-
linux-x86_64 \
  -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
docker compose version
```

21. Create a project folder and compose file:

```
mkdir ~/compose-lab && cd ~/compose-lab
```

```
cat > docker-compose.yml << 'EOF'
services:
  web:
    image: nginx:alpine
    ports:
      - "8081:80"
    depends_on:
      - cache
  cache:
    image: redis:alpine
EOF
```

22. Start both services in the background:

```
docker compose up -d
```

23. Check both containers are running:

```
docker compose ps
```

Browse to `http://<ec2-public-ip>:8081` to confirm the web service responds. The cache service (Redis) runs privately on the same Compose network, reachable from web at hostname `cache`.

```
docker compose logs      # view logs from both services
docker compose down      # stop and remove both containers
```

Lab 4 — Persisting Data with Volumes

Goal: prove that data survives container removal when stored in a Docker volume, and is lost when it isn't.

24. Create a named volume:

```
docker volume create app-data
```

25. Run a container that writes a file into the volume:

```
docker run -d --name writer -v app-data:/data busybox sh -c "echo 'hello from volume' > /data/message.txt && sleep 3600"
```

26. Remove the container (but not the volume):

```
docker rm -f writer
```

27. Start a brand-new container using the same volume and read the file back:

```
docker run --rm -v app-data:/data busybox cat /data/message.txt
```

You should see "hello from volume" printed — proof the data persisted across containers, because it lived in the volume, not inside any single container's writable layer.

28. Contrast this with data written without a volume:

```
docker run -d --name no-volume busybox sh -c "echo 'temporary data' > /tmp/note.txt && sleep 3600"
docker rm -f no-volume
docker run --rm busybox cat /tmp/note.txt    # file will NOT exist — new container, no shared storage
```

```
docker volume ls          # list volumes
docker volume rm app-data  # clean up when done
```

Clean Up

To avoid ongoing AWS charges once you're done with all four labs:

29. Stop any remaining containers: `docker stop $(docker ps -q)`
30. In the EC2 console, select the instance.
31. Click Instance state → Terminate instance, then confirm.

Summary

Docker packages applications into lightweight, portable containers that share the host OS kernel, making them faster and cheaper to run than full virtual machines. Across these four labs on a single EC2 instance, you: ran prebuilt containers, built and ran your own custom image from a Dockerfile, orchestrated multiple connected services with Docker Compose, and used volumes to persist data beyond a container's lifecycle — the same core workflow used to deploy containerized applications in production on AWS.